# School of Informatics & Cybersecurity - Assessment Submission Coversheet

## Student Information

| | |
|---|---|
| Student Number: | B00170371 |
| Student Name: | Alexandru Belecci |

## Module Information

| | |
|---|---|
| Module Title: | Security Architecture |
| Module Code: | COMP H2701 |

## Assessment Information

| | |
|---|---|
| Assessment Title: | Designing and Implementing a Proof of Concept Cloud Solution based on a client's needs |

## Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

☑ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

| | |
|---|---|
| **Student signature:** | Belecci |
| **Date:** | 30/04/2026 |

# Content

Introduction ..... 3

AWS Architecture Diagram ..... 4

Proof of Concept Implementation .....8

Justification of Additional Services .....21

Sustainability .....23

Conclusion .....24

References .....25

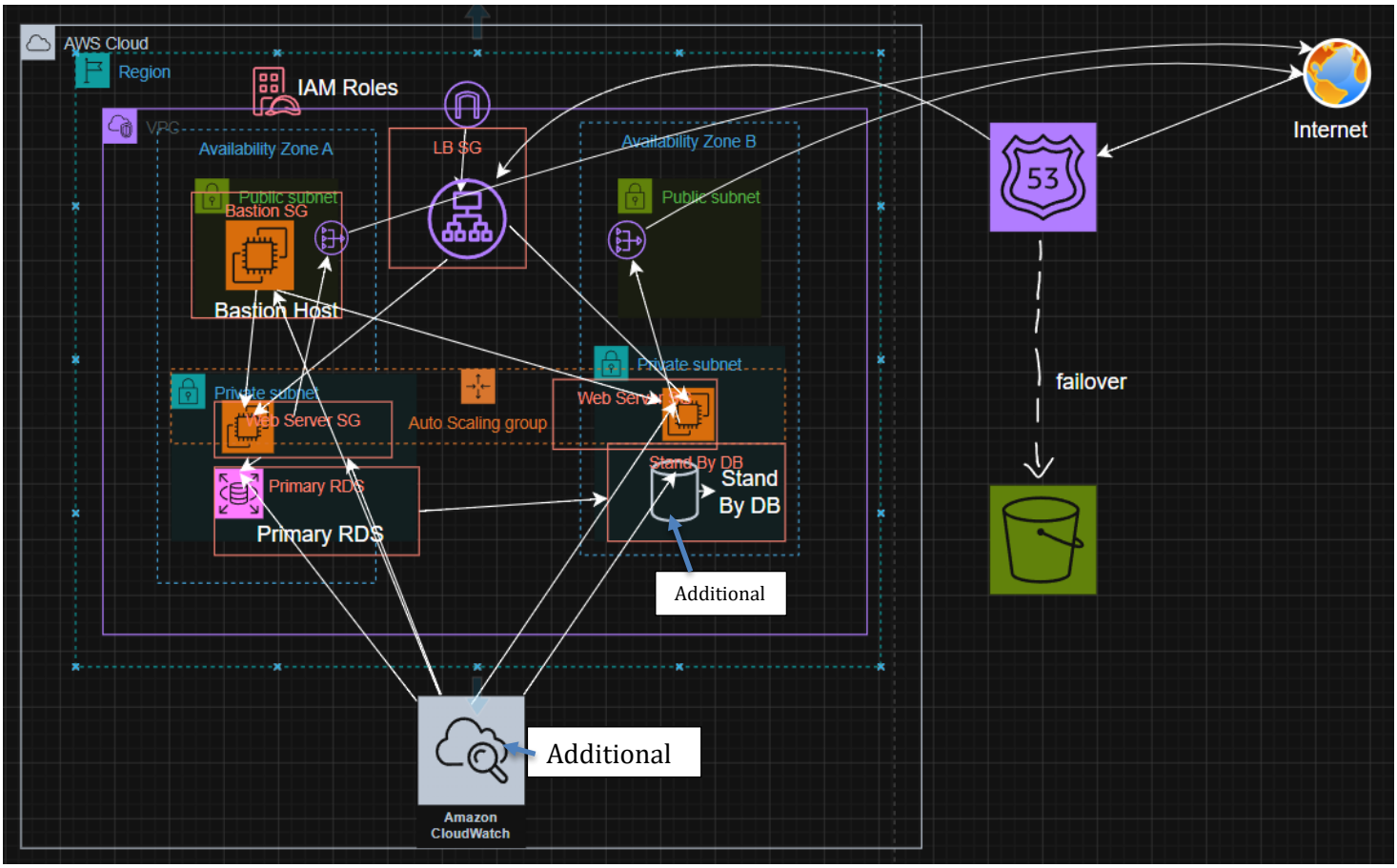
# Introduction

This report is based on improvement and migration of security architecture posture of a small digital startup 'WinLocal' to AWS Cloud Environment. This startup is connecting local businesses and customers through giveaways and competitions, and its popularity is growing fast and the business is expanding.

The current on-premises infrastructure is not sufficient for the growing business as system weaknesses are revealed because of the massive user traffic. The weaknesses include low availability, lack of scalability, a serious security breach and poor database reliability. The ransomware incident that resulted in user data compromission which remained undetected for 10 weeks, and this speaks a lot about system security and monitoring.

Aligned with AWS Well-Architected Framework, the report will cover AWS architecture design, PoC and 2 extra services implementations for addressing WinLoca's security weaknesses maintaining cost optimization and following sustainable practices.

# AWS Architecture Diagram




## Legend

IAM Roles

EC2 Instance: Bastion Host, Web Server 1, Web Server 2

Public Subnet

Private Subnet

Primary RDS

Auto Scaling Group

Internet Gateway

 NAT Gateway



Security Groups

 S3 Bucket

 Load Balancer

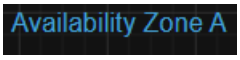


Route 53

 Internet

 Stand By Database (Additional not in PoC)

 CloudWatch (Additional not in PoC)



Availability Zone

 VPC

 Region

 AWS Cloud

*Overview*

A complete architecture on a cloud-based system which is secure and provides the required availability of five nines and scalability. Designed to address the problem of the WinLocal system's infrastructure and moving it to the AWS cloud.

*Networking Layer*

In the same region in the AWS Cloud across 2 Availability Zones (A and B) for elimination of single points of failures and achieving the required availability of 99.999% the VPC is deployed. Within the VPC, 4 subnets were created, 2 public for hosting the Bastion Host for accessing the Web Servers and Databases from the private subnets, and the Load Balancer for managing the traffic flows coming from the internet through the Internet Gateway. While through NAT Gateways that sit in the public subnets the EC2 instances from the private subnets access the internet for updates required without internet exposure.

*Compute Layer*

In both private subnets 1 EC2 instance that serves as the Web Servers run, for hosting the WinLocal's website. They do not run simultaneously as the Auto Scaling Group is scaling as needed, so during normal periods only one instance is running and the second one only on demand, removing the cost of keeping 2 at a time and the need to manually intervene. The Auto Scaling Group is configured to run a minimum of 1 instance and maximum of 2. (Issue 3 address)

*Database Layer*

In both private subnets of each AZ a Database is deployed, in AZ-A the Amazon RDS which serves as the primary DB for all data manipulation. A replacement for the old Microsoft SQL Server, which is easily manageable and scalable. (Issue 2 address)

In AZ-B a 'Stand By copy' of the primary RDS is available in case of any failure of it. Multi AZ RDS Configuration, Not in PoC and will be further justified in Task 3.

*Load Balancer*

The Application Load Balancer sits in between both the public subnets, having a single job 'Distributing the internet traffic' between Instances from each Availability Zone. In case of failure of the Load Balancer or become unavailable, the DNS service which we are using Route 53 will redirect the traffic to a static webpage that is hosting on S3 bucket, displaying a relevant message. (Issue 1 address)

*Security Layer*

For enforcement of least privilege access control techniques to each component of the Architecture a Security Group is applied limiting the entry points. Working together with the IAM Roles for permission controls accessing any AWS resource for insurance that unauthorized access will be prohibited and granted only required rights.

*Monitoring*

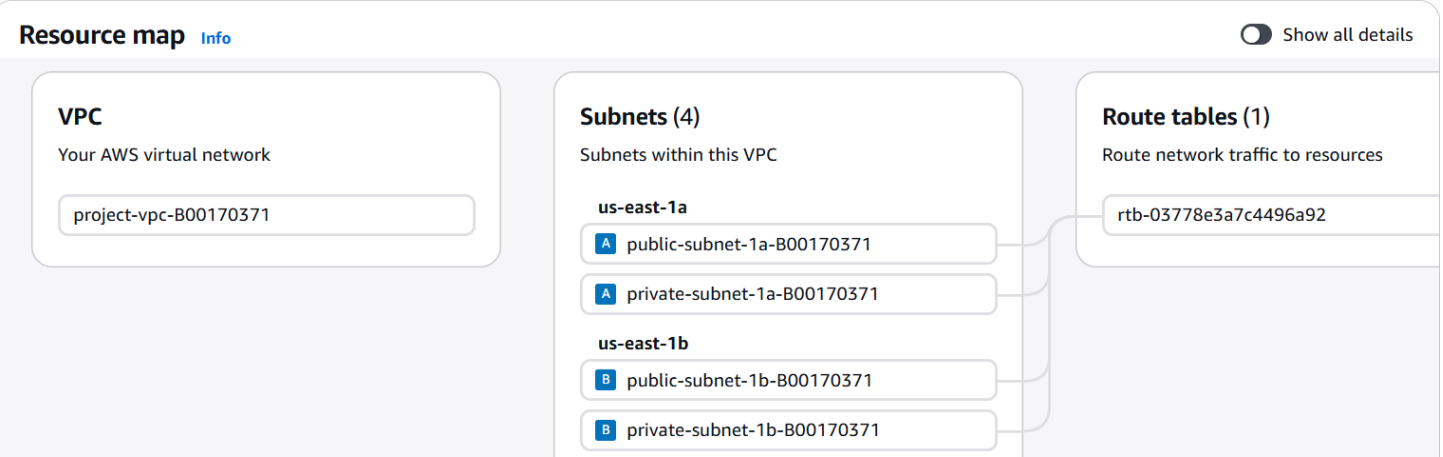
Not in PoC and will be further justified in Task 3.

For real-time monitoring of the system's behaviour and alertion across resourcess, CloudWatch is the service.

Proof of Concept Implementation

## *Network*

### *VPC*

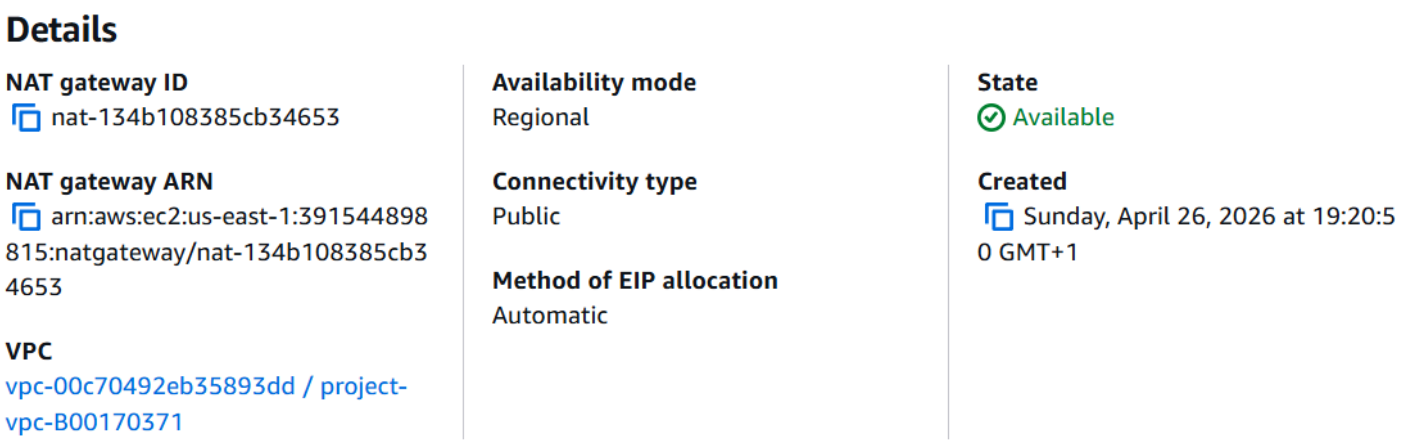



WinLocal's VPC was created under the name of *project-vpc-B00170371* with the CIDR Block of 10.0.0.0/16 giving it space for future growth. The VPC was launched across 2 AZs in the us-east-1 region (a and b) for assurance of high availability addressing the 1st issue faced by WinLocal, the periodic failure. With 2 subnets in each AZ, private and public subnets. The public subnets (10.0.1.0/24 and 10.0.3.0/24) will host the Bastion instance in AZ A and Load Balancer in AZ B, both facing the internet. The private subnets (10.0.2.0/24 and 10.0.4.0/24) will host Web Servers and Databases, restricting direct access from the internet.

*Internet/NAT Gateway*

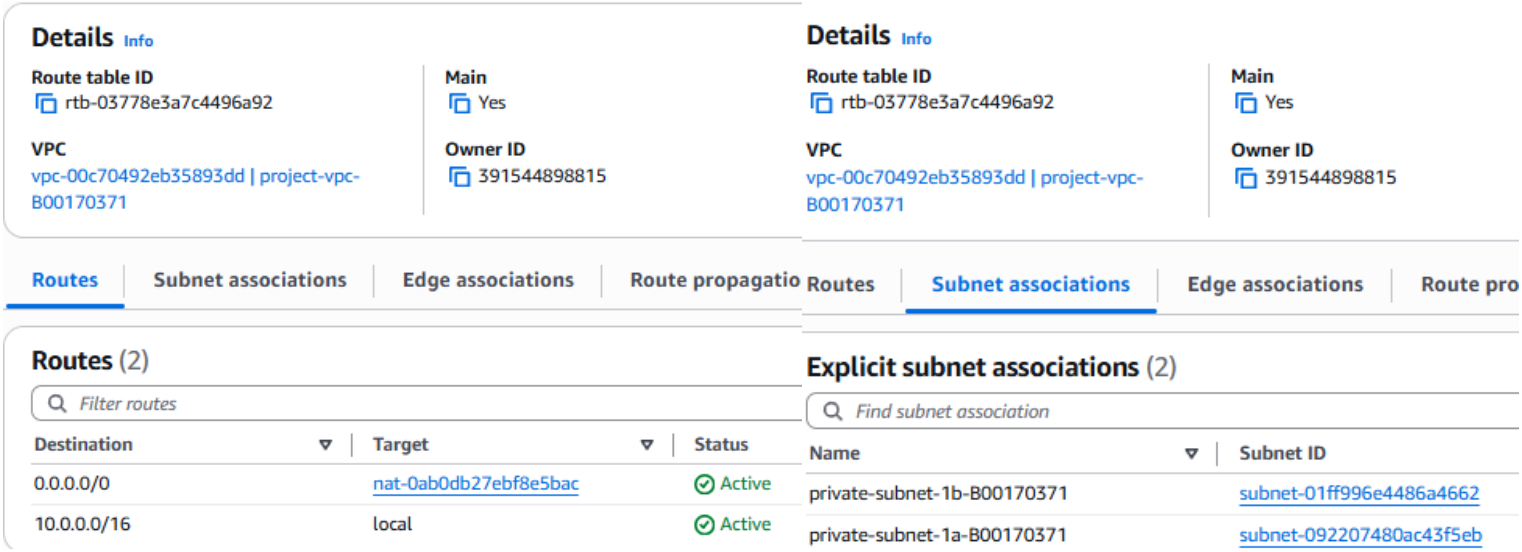



The created Internet Gateway (net-gateway-B00170371) was successfully attached to our project VPC. The importance of Internet Gateway is to allow the outside traffic to access the website, serves the entry point.

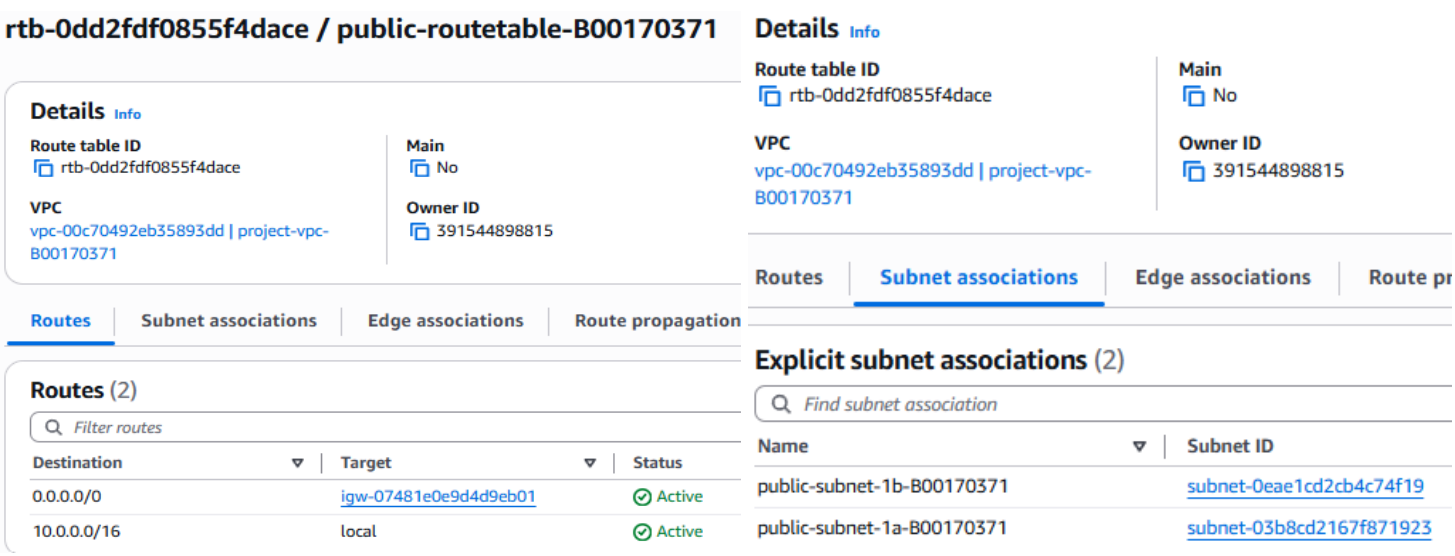



The state: Available means that NAT Gateway successfully deployed and connected to the project VPC created earlier on. Public connectivity was implemented to allow our instances to reach the internet for patching and updates, most suitable for WinLocal's private instances that should not be exposed to the internet.

*Route Tables*



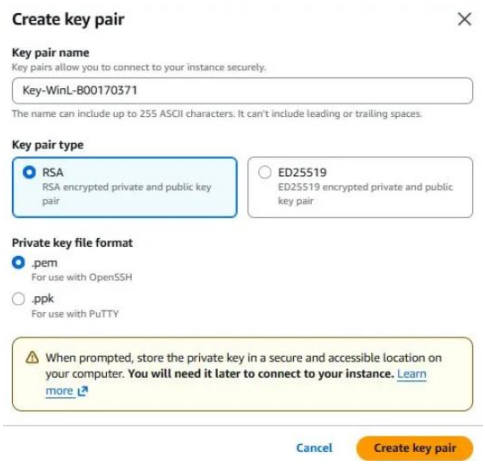

The private Route Table is associated with the private subnets (private-..-1a/1b), it directs the traffic from private instances through the NAT Gateway, giving an improvement to WInLocal's flat architecture.

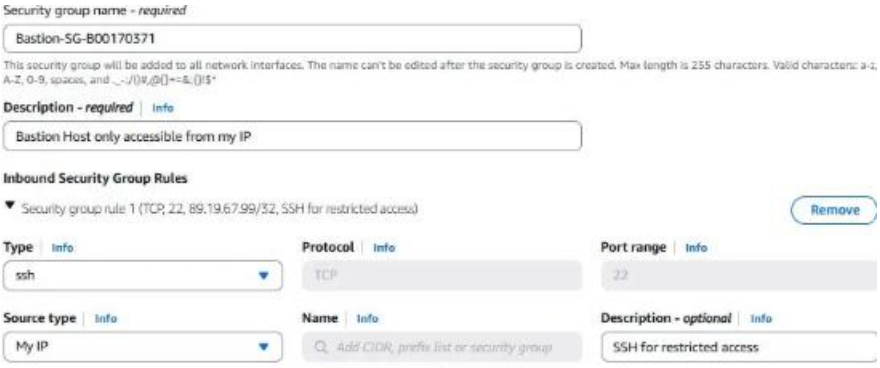



The public Route Table is associated with the public subnets (public-..-1a/1b), which allows the internet traffic to reach out to the public instances (Bastion Host and Load Balancer).

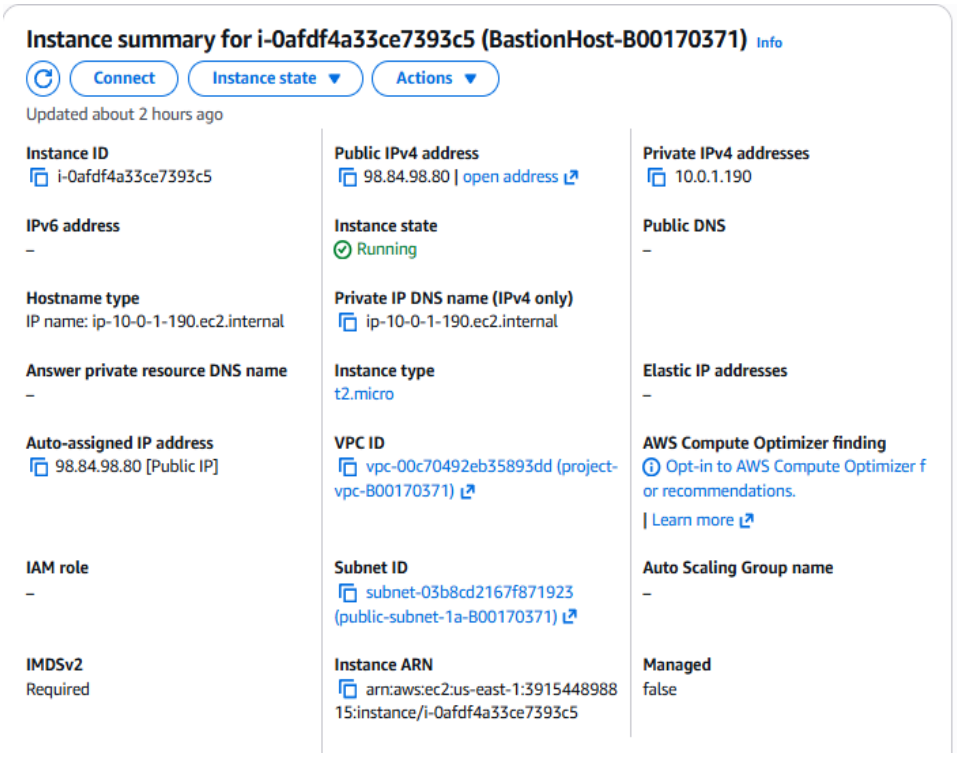
## *EC2 Instances*

### *Bastion Host*

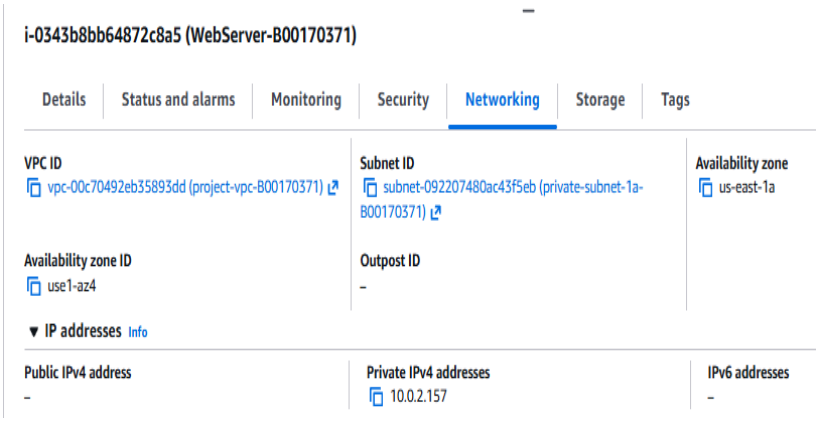


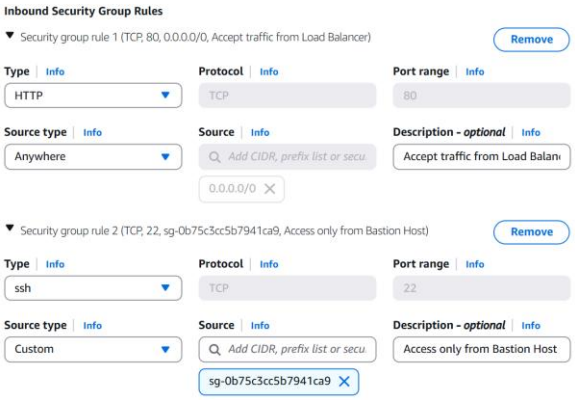




A new private key for this EC2 instance was created 'Key-WinL-B00170371' that will prove my authentication when accessing it via SSH, which will be the only way of access as the Security Group created with it 'Bastion-SG-B00170371' restricts SSH traffic from my IP only, also will be the only access further to other instances in the private subnets. The attack surface is minimized, and Issue number 4 is addressed where the ransomware attack was a consequence of weak access control.



*Web Server*



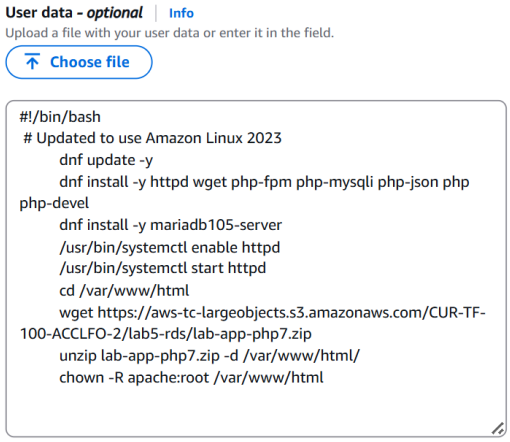



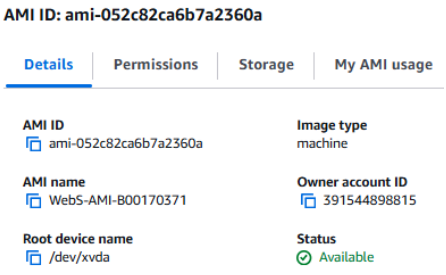

In the project's VPC the Web Server instance deployed in AZ A's private subnet gained the private IP: 10.0.2.157 and no public address, because it is not supposed to be reachable from the open internet. The security group allows HTTP traffic on port 80 from anywhere and restricts SSH access only to Bastion Host instance settled earlier. (Issue 4 address)

User Data script is automatically triggered when EC2 instances launch and get auto configuration. Installing:

- Apache
- PHP
- SQL

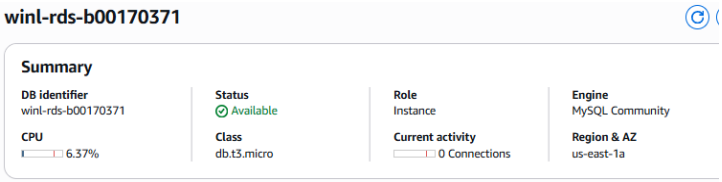
Addressing the issue number 3 'Manual Intervention'.



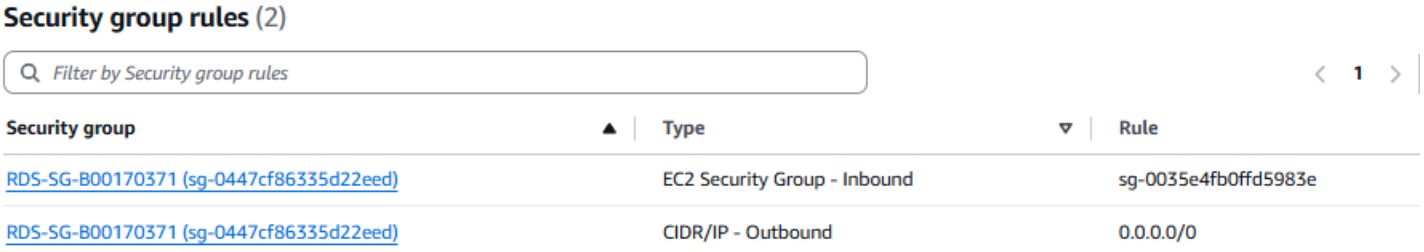




The AMI snapshot of the Web Server was successfully created and gained the Status: Available so if the Auto Scaler requires another Web Server instance, it will launch a complete copy of it during high volume traffic eg. Campaigns.

*RDS MySQL*




RDS instance 'winl-rds-b00170371' successfully deployed in a single-AZ us-east-1a and running on db.t3.micro. It will replace WinLocal's current Microsoft SQL Server which regularly was running out of space addressing Issue 2.

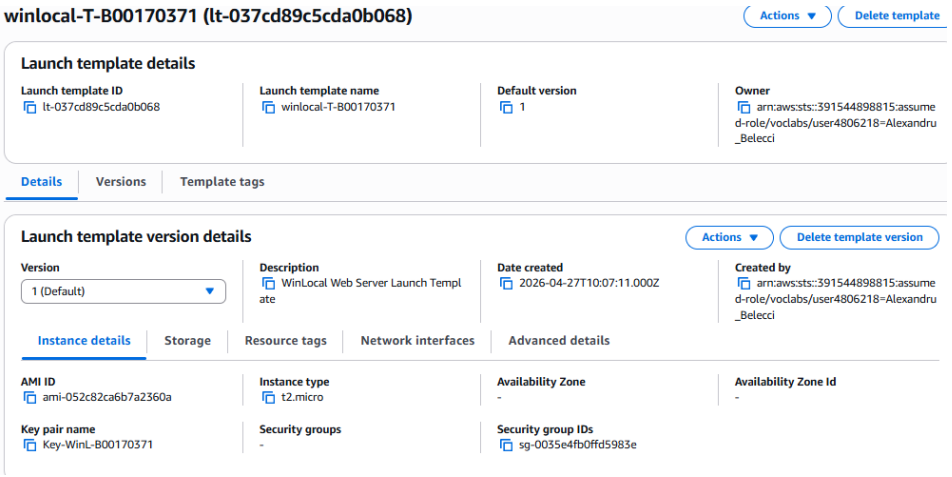



Access is restricted only from Web Servers which is also improving the security posture of the system. (Issue 4)

*Launch Template*

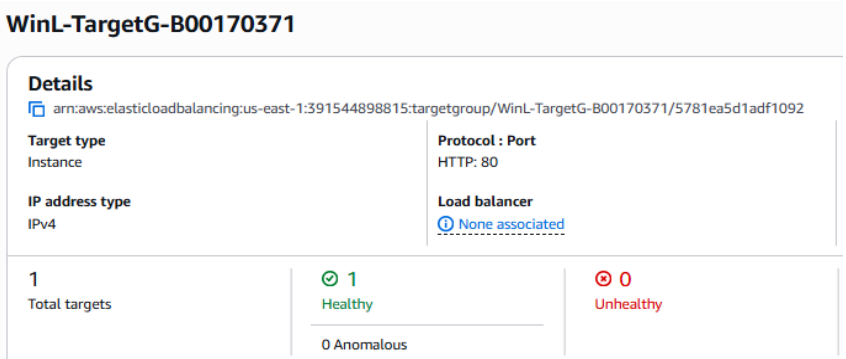
The template winlocal-T-B00170371is the blue print of each EC2 instance that Auto Scaler will launch, configurations pre-settled:

- AMI – Web Server AMI
- Type – t2.micro
- Security Group – WebS SG
- Key Pair – for SSH Control Key-WinL-B00170371



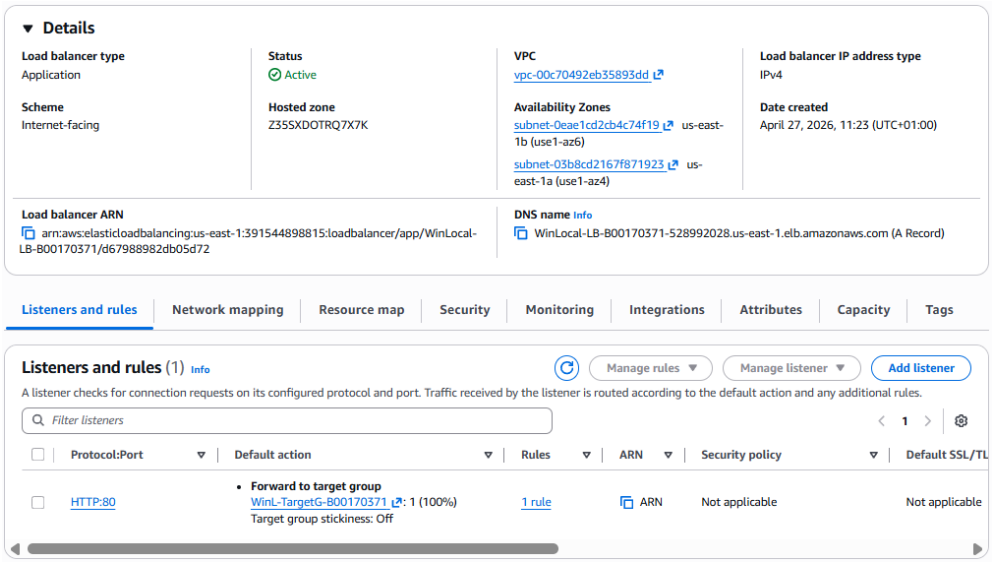

Which will be triggered when for launching of identical instances, when limits are hit. (Issue 3)

*Target Group*




1 healthy instance is shown by the Target Group, so far only one instance is created and is healthy. Based on this the Load Balancer will send traffic to the Target Group and it will redirect it to the healthy instance.

*Load Balancer*



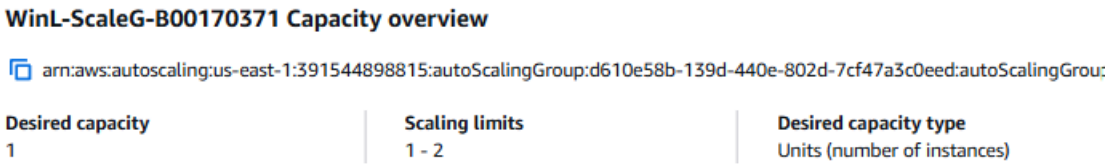

Target Group.

The only entry point to access the WebSite, which is a replacement for the old Netscaler used by WinLocal.

Traffic is never passed directlu to website, Load Balancer makes sure that traffic is sent to healthy instances. Ensuring of availability. (Issue 1 address)

The LB is active and is facing the internet, listens on port 80 and is forwarding traffic to the

*Scaling Group + Scaling Policy*

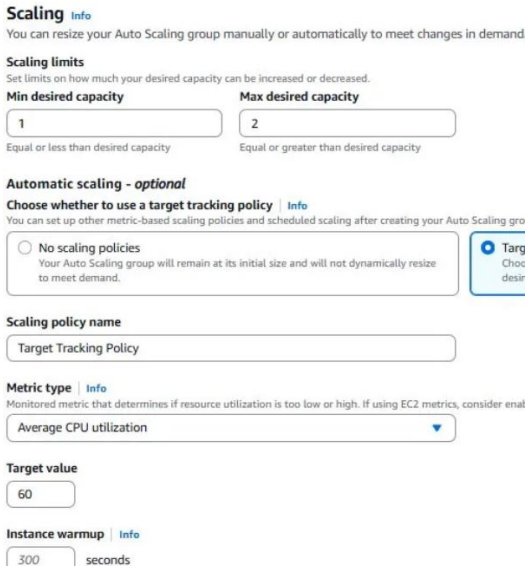



The Scaling Group was configured with minimum capacity 1 and maximum capacity 2 each instance from AZs, desired capacity set to 1.

The Auto Scaling Group is an important instance, as it scales the number of instances based on demand saving resources and clients' money. (Issue 3 address)

During Campaigns the number of instances will increase from 1 to 2.

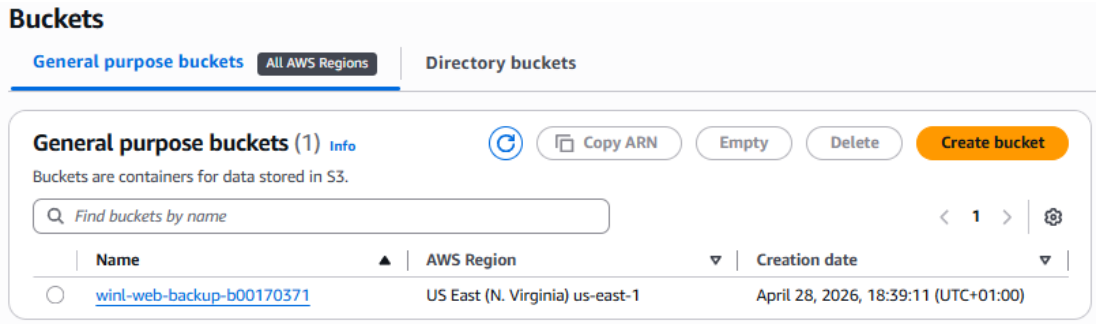
For maintaining the CPU's utilization for each instance at 60%, and a warmup time of 300 seconds, a target Tracking Policy is implemented. Which will only launch new instances once the existing overloads, keeping the website always available and 300 warm seconds accords instances time to become functional.



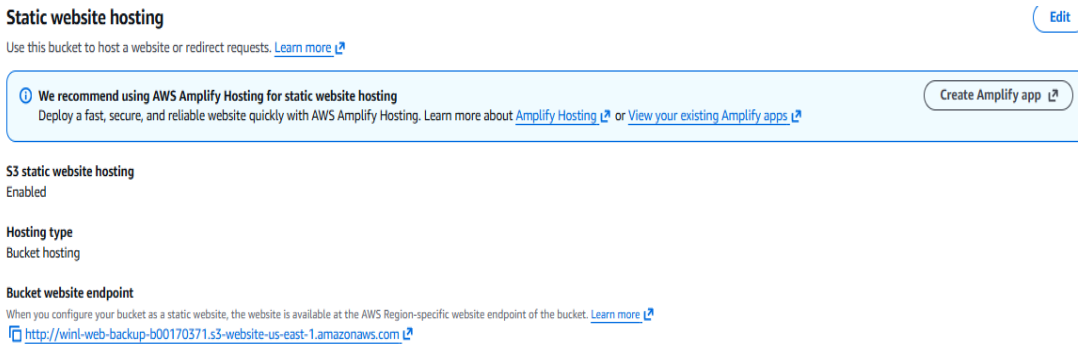
*S3*

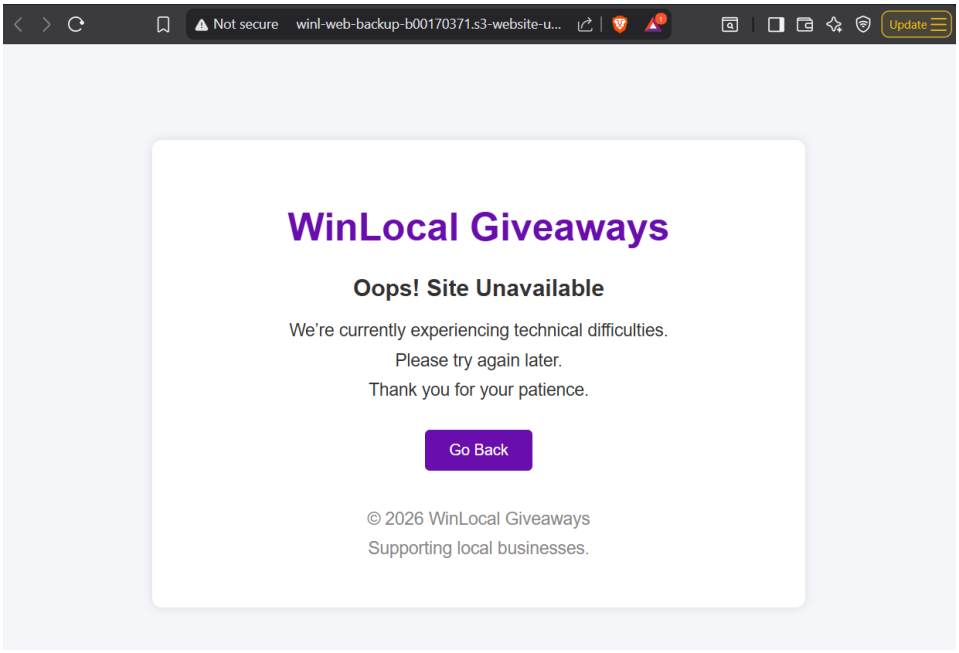
*Static Backup Web Site*

The S3 bucket was created to host the static failover web page. So, in case of failure of web site, Route 53 will redirect the traffic flow to S3 web static page via DNS failover mechanism. Made it public for access, and will automatically activate when Web Server instances will become unavailable.




The activation of S3 was successful, enabled the website hosting for customers static webpage display.



*Bastion Access*

```
C:\Users\Admin\OneDrive - Technological University Dublin\Cyber Year 2\
Sem2\Security Architecture\project>ssh -i "Key-WinL-B00170371.pem" ec2-
user@50.19.130.72
The authenticity of host '50.19.130.72 (50.19.130.72)' can't be establi
shed.
ED25519 key fingerprint is SHA256:RQUsN9xmD9962QAcvVLRovTQQxrQ0FSfX8Gix
Y58W4Y.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? ye
s
Warning: Permanently added '50.19.130.72' (ED25519) to the list of know
n hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-1-190 ~]$
```

Proof of successful connection to our Bastion Host through SSH, used for a secure entry point in the system. This method of entry point is reducing the risk of unauthorized access. As private instances do not require public IP addresses.

```
PS C:\Users\Admin\OneDrive - Technological University Dublin\Cyber Year 2\Sem2\Security Architecture\project> scp -i "Ke
y-WinL-B00170371.pem" "Key-WinL-B00170371.pem" ec2-user@50.19.130.72:~/.ssh/
Key-WinL-B00170371.pem                                        100% 1674    13.7KB/s   00:00
```

Next, once inside the Bastion Host we should transfer the private key used for connection to it for future connection to servers.

*Server Access*

```
[ec2-user@ip-10-0-1-190 ~]$ ssh -i "~/.ssh/Key-WinL-B00170371.pem" ec2-
user@10.0.2.157
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-2-157 ~]$
```

And now we are inside the server.

*Database Access*

```
[ec2-user@ip-10-0-2-157 ~]$ mysql -h winl-rds-b00170371.cxo8jg6cvkrr.us
-east-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 1705
Server version: 8.4.8 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input sta
tement.

MySQL [(none)]>
```

And successfully connected to the Database.

Next a database 'winlocal' was created in the RDS instance. In the DB table 'users' created for future customers stocking with basic details:

- ID
- Name
- Address

At the end sample data was inserted for simulation.

```
MySQL [(none)]> CREATE DATABASE winlocal;
Query OK, 1 row affected (0.032 sec)

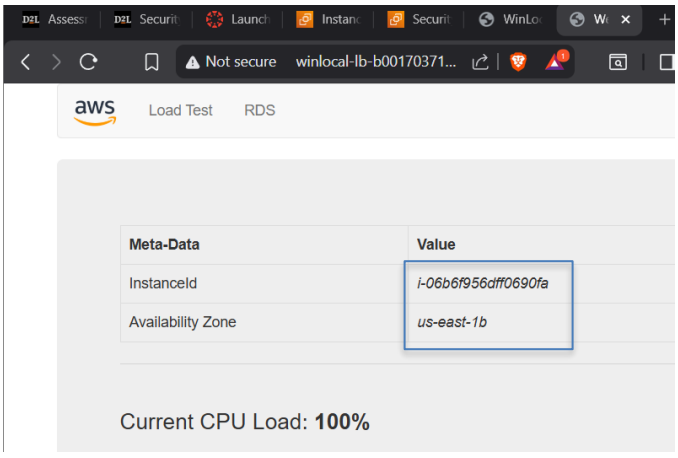
MySQL [(none)]> USE winlocal;
Database changed
MySQL [winlocal]>
MySQL [winlocal]> CREATE TABLE users (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     name VARCHAR(50),
    ->     address VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.068 sec)

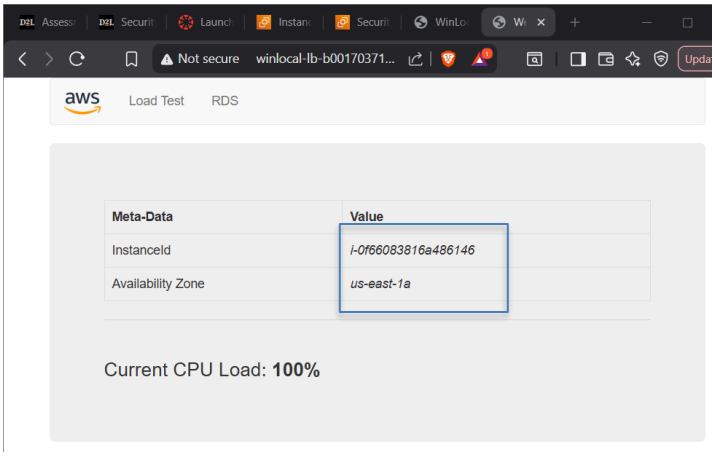
MySQL [winlocal]>
MySQL [winlocal]> INSERT INTO users (name, address)
    -> VALUES ('John Doe', 'Dublin');
Query OK, 1 row affected (0.015 sec)
```

```
[ec2-user@ip-10-0-2-157 ~]$ curl http://localhost
John Doe - Dublin<br>[ec2-user@ip-10-0-2-157 ~]$
```

Which is successfully output, meaning that web server is running and accessible locally. Screenshot confirms functionality of the application that is operating in the private subnet.

*Load Balancer and Auto Scaling Group in Action*




These 2 screenshots are proof of both Load Balancer and Auto Scaling Group working.

- As the load increased, breaking the 60% trigger set for the Auto Scaler, it automatically launched the 2nd web instance for demand handling.
- Then Load Balancer dynamically dividing the flow between 1st instance and 2nd instance, as every time the page was refreshed the Instance ID changed between the 2 available instances which reside in 2 different AZs - A and B.

This is an assurance that during busy periods, the application will remain available to public with no failures.

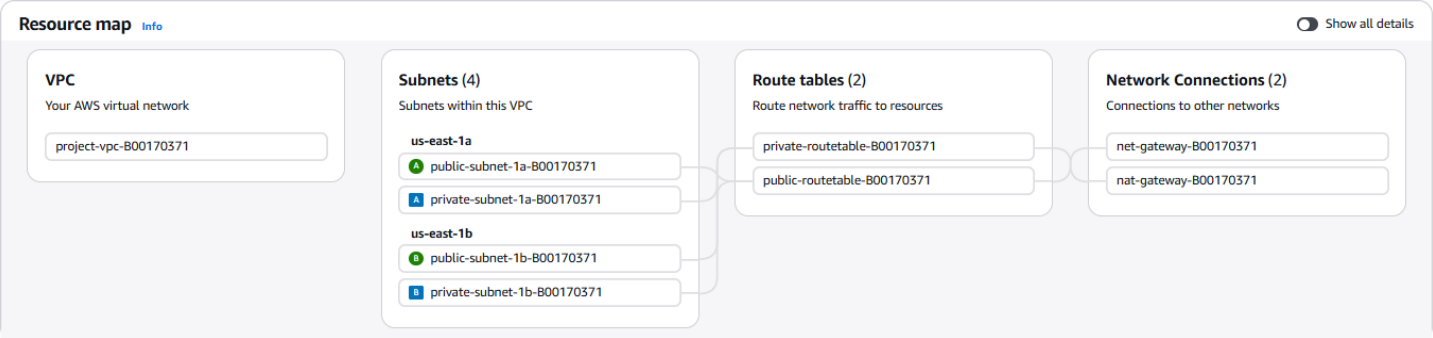
*Security Groups*

| Security group ID | Security group name | VPC ID | Description |
|---|---|---|---|
| sg-0b75c3cc5b7941ca9 | Bastion-SG-B00170371 | vpc-00c70492eb35893dd | Bastion Host only accessible from my IP |
| sg-0035e4fb0ffd5983e | WebServer-SG-B00170371 | vpc-00c70492eb35893dd | launch-wizard-1 created 2026-04-26T2... |
| sg-0447cf86335d22eed | RDS-SG-B00170371 | vpc-00c70492eb35893dd | Created by RDS management console |
| sg-04c6023bd31baf7bc | LB-SG-B00170371 | vpc-00c70492eb35893dd | Accept traffic to load balancer |

All security groups created within the VPC for each instance following security approach.

- Bastion-SG: Limited access via SSH.
- WebServer-SG: Limited access from Bastion Host via SSH and accepts traffic from Load Balancer.
- RDS-SG: Only accepts traffic from Web Server's SG on MySQL's port 3306
- LB-SG: Accepting traffic coming from open Internet.

*Resource Map*

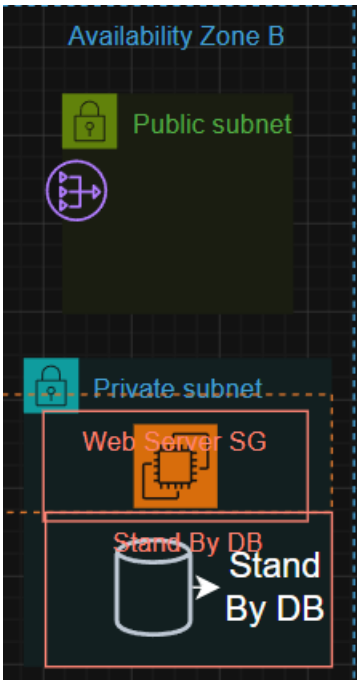



End resource map shows everything that was implemented.

- VPC: project-vpc-B00170371
- Subnets: 2 public/private in 2 AZs
- Route Tables: Private/Public
- Connections: NAT/Internet Gateway

## Services Beyond PoC

*Multi-AZ Stand By Database*



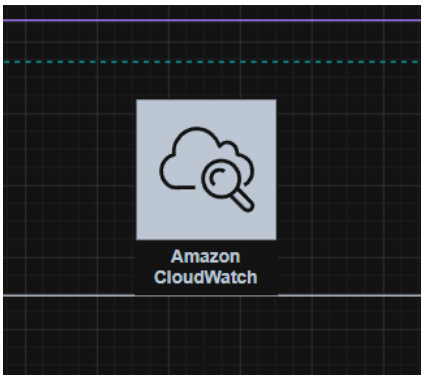

As outlined in the brief a key requirement is the achievement of high availability (target > 99.999%). Therefore, the deployment of a Multi-AZ RDS will support this implementation and will ensure fault tolerance of the application (2 Databases).

Multi-AZ Database deployment > creation of the original DB's replica in the second Availability Zone, called Stand By Database.

The failure of the main database will not cause the application to stop, as the second stand by database that is ready to take over any time, addresses Issue #2 – Database outages and storage limitations.
Resulting that best practices of a secure and highly available cloud system is followed, where no access to any data will be lost.

The only reason of not implementing this method in our PoC is the budget limitations, but for future development this should be considered as the data flow will only increase same as the risk of failures.

*CloudWatch*




Monitoring is key, and CloudWatch is the observatory building in the cloud. As if proper system monitoring was implemented, the chances of the ransomware incident remaining undetected would've dropped significantly (Issue 4).

CloudWatch's job is collection of logs, metrics and events which trigger alerts on behaviour:

- CPU's utilization
- System Logs
- Network Activity

Real-Time monitoring will help administrators in detection of anomalies, further investigations and Incident Response/Recovery. Receiving Alerts performed by CloudWatch will slightly remove the pressure from WinLocal's security team, providing continuous visibility.

## Sustainability

Minimization of environmental impact is key, which is the focus of AWS Well-Architected Framework and our new implementations are following. Also leaving room for improvement for future WinLocal's growth.

Main decision was the **migration to AWS cloud solutions**, which is running more efficiently than the traditional on-premises setups that WinLocal is currently dealing with. As AWS data centers are highly energy efficient which results in lower environmental impact.

**Auto Scaling solution** is great because is saving customers money but also is important as it reduces energy waste through running instances only on demand, which provides a better solution for WinLocal's experience of variance in demand. Avoiding the Idle run of the systems instances is vastly reducing the impact to environment, and only involving new instances as needed.

**Instance sizing** is helping nearly in same manner as scaling solution by providing the needed sizes of instances avoiding energy waste and cost of it, eliminating over-provisioning for matching demand. In WinLocal's case all instances run on t2/3.micro which is perfect for its load.

**Managed Services,** used for improvement of resource utilization and removing the need to maintain any hardware, in our case RDS which is completely controlled and scaled by AWS. This removes the need for manual equipment holding and intervention.

Also, some future considerations should be taken to account, as specified above CloudWatch implementation but for billing alerts, AWS Cost Explorer for monitoring of used resources, Lifecycle S3 Data Policies.

- CloudWatch Billing Alerts
  Reduction of any unnecessary resource usage through limiting costs and setting alerts if breached.
- AWS Cost Explorer
  Cost and efficiency improvement analysis over the resources.
- Lifecycle S3 Data Policies
  Data movement into lower energy storage, eg. S3 Glacier

# Conclusion

Finally, all issues presented by WinLocal's team were addressed and fixing instructions were provided as well as future considerations to be considered. Solutions proposed to boost up the main requirements for business operation:

- Availability
- Scalability
- Security
- Fault Tolerance

Availability and Fault Tolerance were improved through deployment across 2 AZs eliminating potential single points of failure.

Auto Scaling groups and Load Balancer were handling increased traffic flow, keeping the application running and fully functional.

Security was strengthened with the use of Security Groups, Bastion Host main access and subnet isolation, which highly improved WinLocal's security posture. Reducing internet exposure of sensitive components.

All these are demonstrated in Proof of Concept, where successful deployment of improved infrastructure was performed and tested under high traffic load. Traffic loads did not stop the operation of the application and no failure was monitored. As well as further improvements proposed, it aligns with AWS Sustainability Pillar as major reductions in consumption and resource waste was eliminated which completely satisfies WinLocal's requirements and future achievements.

# References

Academy, AWS. “Cloud Security.” *Instructure.com*, 2022, awsacademy.instructure.com/. Accessed 2026.

AWS. “Security Pillar - AWS Well-Architected Framework - Security Pillar.” *Docs.aws.amazon.com*, 6 Nov. 2024, docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html. Accessed 30 Apr. 2026.

---. “The AWS Security Reference Architecture - AWS Prescriptive Guidance.” *Docs.aws.amazon.com*, docs.aws.amazon.com/prescriptive-guidance/latest/security-reference-architecture/architecture.html. Accessed 29 Apr. 2026.

South, Greg. “Security Architecture COMPH2701.” *Tudublin.ie*, 2026, brightspace.tudublin.ie/d2l/home/523174. Accessed 2026.